Security Audit Report

Avail Cosmos and Backend Changes

v1.0

January 15, 2026

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 9
Functionality Overview 9
How to Read This Report 10
Code Quality Criteria 11
Summary of Findings 12
Detailed Findings 14
1. Malicious vote extensions can arbitrarily mutate bridge state after late precommit injection 14
2. ProcessProposal accepts all proposals without validating vote extensions 15
3. Non-deterministic time check causes consensus failures 15
4. Only the latest 100 settlements are retrieved without marking them as completed, which prevents processing earlier settlements 16
5. Refund logic allows already fulfilled requests to be reimbursed, allowing double spends and prematurely refunded deposits, resulting in locked liquidity 16
6. Unvalidated RFF publication enables P2P network spam and resource exhaustion 17
7. Unbounded pending settlements are processed during the PreBlocker, which can cause consensus delays 18
8. External RPC dependencies in consensus logic allow validators to be stalled 19
9. Missing minimum block confirmation margin allows external reorgs to invalidate verified requests 20
10. Fetching vote extension events without timeouts can stall consensus 20
11. Unsupported settlement chain stops CA processing for all settlements 21
12. Disabling libp2p limits enables remote DoS 21
13. Transient DSG send failures slip through retries and stall key operations 22
14. Replayed DSG messages crash CA nodes through closed channel writes 22
15. Incomplete fee grant implementation 23
16. Switching the connected wallet continues to use the previous EVM address 24
17. Compromised SIWE signature enables permanent theft of derived Cosmos and ephemeral wallets 24
18. CreateProtocolFees lacks an authorization check, allowing anyone to set themselves as admin 25
19. Signature verification failure enables zero solver addresses and burns settlements 26
20. Using log.Fatal can halt the validator 26

21. Known vulnerable Go dependencies 27
22. Gas refund flag in deposit packet is ignored by the Cosmos chain 27
23. Default price of 1.0 for unknown tokens 28
24. Signed batched call signatures never expire, enabling replay when leaked 28
25. Params validate performs no validation, allowing invalid parameters to be stored 29
26. Protocol fees messages lack validation for fee configuration fields 29
27. Service startup might hang due to an infinite peer connection retry loop 30
28. DSG threshold set to 100% 30
29. Potentially non-deterministic nonce derivation 31
30. Missing disabled vote extension check 31
31. Multiple messages return empty responses 32
32. Missing validation for RPC URLs and vault contract addresses during initialization 33
33. Hardcoded gas limits may cause inefficiency or failures 33

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Avail Technology Ltd to perform a security audit of Avail Cosmos and Backend Changes.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following targets:

Repository	https://github.com/arcana-network/go-ethereum
Label	Paths referencing this target are prefixed below with `go-ethereum:`
Scope	The scope is restricted to the changes introduced in the following git commits • `9cae2798d031e993b2fc915240b77c835ce2c936` • `86a0bc7db5b3d92f3c394cc24dad184c2b6b8a4f` • `d52a092bed9a4b950d9dbc46404a61d00ad4848e`

Repository	https://github.com/availproject/xarchain
Label	Paths referencing this target are prefixed below with `xarchain:`
Commit	`5e6d214a8ea28be382673aa5ccec94c0681f1255`
Scope	The scope is restricted to the files in the `x/chainabstraction` directory, excluding the auto-generated code in the module, simulation, and types directories.
Fixes verified at commit	`d5bcfc1e46bfc7dd22db9e92e289dcda8be76efd` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Repository	https://github.com/availproject/ca-node
Label	Paths referencing this target are prefixed below with `ca-node:`
Commit	`f86f21c1c03388eed8b25e29d0fca50476427fed`
Scope	The scope is restricted to the following directories: • `internal/cosmos` • `internal/p2p` • `vservice`
Fixes verified at commit	`b24c889a78b9acea3c874c7cac77aaff5f9184a2` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Repository	https://github.com/availproject/nexus-sdk
Label	Paths referencing this target are prefixed below with `nexus-sdk:`
Commit	`183e731477a147703aba70058ba4cad3b3c55cd1`
Scope	The scope is restricted to the following code:

	• Sign/key logic implemented in the `_createCosmosWallet` function in `packages/core/sdk/ca-base/ca.ts#L415-L428` • `packages/core/sdk/ca-base/swap/sbc.ts`
Fixes verified at commit	`247b7ad579ae3e1fcb1bc8e3d956523014490c7e` Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Avail Nexus is a meta-interoperability protocol that eliminates blockchain fragmentation by connecting liquidity, assets, and coordination logic at the base layer. It abstracts complexities such as manual bridging, chain switching, wallet switching, swaps, and complex approvals for the end user to create a seamless "bridgeless" experience.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium-High	The codebase comprises several interconnected components, including the Cosmos SDK appchain, an offchain node responsible for fulfilling bridging requests and settlements across chains, a Typescript SDK, and a vault smart contract.
Code readability and clarity	Medium-High	-
Level of documentation	Medium-High	The client provided detailed documentation.
Test coverage	Low	The code is largely untested, resulting in poor code test coverage.

Summary of Findings

No	Description	Severity	Status
1	Malicious vote extensions can arbitrarily mutate bridge state after late precommit injection	Critical	Resolved
2	`ProcessProposal` accepts all proposals without validating vote extensions	Critical	Resolved
3	Non-deterministic time check causes consensus failures	Critical	Resolved
4	Only the latest 100 settlements are retrieved without marking them as completed, which prevents processing earlier settlements	Critical	Acknowledged
5	Refund logic allows already fulfilled requests to be reimbursed, allowing double spends and prematurely refunded deposits, resulting in locked liquidity	Critical	Partially Resolved
6	Unvalidated RFF publication enables P2P network spam and resource exhaustion	Major	Acknowledged
7	Unbounded pending settlements are processed during the `PreBlocker`, which can cause consensus delays	Major	Acknowledged
8	External RPC dependencies in consensus logic allow validators to be stalled	Major	Acknowledged
9	Missing minimum block confirmation margin allows external reorgs to invalidate verified requests	Major	Acknowledged
10	Fetching vote extension events without timeouts can stall consensus	Major	Resolved
11	Unsupported settlement chain stops CA processing for all settlements	Major	Resolved
12	Disabling libp2p limits enables remote DoS	Major	Resolved
13	Transient DSG send failures slip through retries and stall key operations	Major	Resolved
14	Replayed DSG messages crash CA nodes through closed channel writes	Major	Resolved
15	Incomplete fee grant implementation	Major	Acknowledged

16	Switching the connected wallet continues to use the previous EVM address	Major	Resolved
17	Compromised SIWE signature enables permanent theft of derived Cosmos and ephemeral wallets	Major	Acknowledged
18	`CreateProtocolFees` lacks an authorization check, allowing anyone to set themselves as admin	Major	Resolved
19	Signature verification failure enables zero solver addresses and burns settlements	Minor	Resolved
20	Using `log.Fatal` can halt the validator	Minor	Acknowledged
21	Known vulnerable Go dependencies	Minor	Resolved
22	Gas refund flag in deposit packet is ignored by the Cosmos chain	Minor	Resolved
23	Default price of 1.0 for unknown tokens	Minor	Resolved
24	Signed batched call signatures never expire, enabling replay when leaked	Minor	Resolved
25	Params validate performs no validation, allowing invalid parameters to be stored	Minor	Acknowledged
26	Protocol fees messages lack validation for fee configuration fields	Minor	Resolved
27	Service startup might hang due to an infinite peer connection retry loop	Minor	Resolved
28	DSG threshold set to 100%	Minor	Resolved
29	Potentially non-deterministic nonce derivation	Minor	Resolved
30	Missing disabled vote extension check	Informational	Resolved
31	Multiple messages return empty responses	Informational	Acknowledged
32	Missing validation for RPC URLs and vault contract addresses during initialization	Informational	Acknowledged
33	Hardcoded gas limits may cause inefficiency or failures	Informational	Acknowledged

Detailed Findings

1. Malicious vote extensions can arbitrarily mutate bridge state after late precommit injection

Severity: Critical

In `xarchain:x/chainabstraction/abci/proposal.go:257-270`, the `computeCA` function that feeds the block proposal selects the vote extension from `ci.Votes[0]` without validating the payload. CometBFT sorts these votes by descending voting power, so the first entry belongs to the validator with the largest stake. Late precommits from prior heights bypass the application's `VerifyVoteExtension` hook, allowing a Byzantine top-power validator to broadcast a forged vote extension after the block is decided. The proposer will decode it, treat its `IntentData` as canonical, and insert the resulting transaction into the block.

Subsequently, the pre-block handler executes those packets via the `keeper.ExecuteVE` function, implemented in `xarchain:x/chainabstraction/keeper/ve_exec.go:56-188` without any verification, directly mutating `RequestForFunds`, `PendingSettlement`, and related bridge state.

This enables arbitrary cross-chain settlements or price updates to be forged and committed, compromising the bridge's integrity. The impact is critical as funds can be misdirected or accounting corrupted without needing a majority coalition.

Recommendation

We recommend validating vote extensions during proposal construction.

Specifically:

- Ensure the `baseapp.ValidateVoteExtensions` function is called before consuming the commit.
- Iterate over all `ExtendedCommitInfo` entries, skip non-`BlockIDFlagCommit` votes, and run the same checks performed in `VerifyVoteExtensionHandler`, rejecting any vote whose intents fail validation.
- Only aggregate intents that pass validation and require consensus (for example, ≥2/3 of voting power submitting an identical payload) before injecting the transaction.
- If no validated payload exists, avoid appending the synthetic transaction so a malicious validator cannot force arbitrary state changes.

Status: Resolved

2. `ProcessProposal` accepts all proposals without validating vote extensions

Severity: Critical

The `ProcessProposal` function in `xarachain:x/chainabstraction/abci/proposal.go:71-102` has all validation logic commented out and unconditionally accepts all proposals. This allows a malicious proposer to inject fake vote extension events (`DepositVEPacket`, `FulfilmentVEPacket`) that never occurred on external chains, manipulate `ExtendedCommitInfo`, and potentially drain funds by creating fake cross-chain events. The commented code includes critical validation via `baseapp.ValidateVoteExtensions` that verifies vote extensions are properly signed by validators and match the `ExtendedCommitInfo`. Without this validation, the bridge's security model is compromised as vote extensions serve as the source of truth for cross-chain events.

Recommendation

We recommend uncommenting and implementing the validation logic, including calling `baseapp.ValidateVoteExtensions` to verify vote extensions are cryptographically signed and match what validators actually committed to.

Status: Resolved

3. Non-deterministic time check causes consensus failures

Severity: Critical

In the `RefundReq` function in `xarachain:x/chainabstraction/keeper/msg_server_refund_req.go:29`, the code uses `time.Now()` to check if a request for funds has expired. This creates non-determinism because each validator will see a different current time based on their system clock, which can differ by milliseconds or seconds. When validators execute the same transaction at different `time.Now()` values, they may reach different conclusions about whether the RFF has expired, causing consensus failures in which the transaction is accepted by some validators and rejected by others. This breaks the fundamental requirement that all validators must execute transactions deterministically and agree on the results. The chain may halt, or transactions may fail intermittently, depending on which validators process them first.

Recommendation

We recommend replacing `time.Now()` with `ctx.BlockTime()`, which provides the deterministic block time that all validators agree on.

Status: Resolved

4. Only the latest 100 settlements are retrieved without marking them as completed, which prevents processing earlier settlements

Severity: Critical

In `ca-node:vservice/settlement_ev.go:44-49` of the `handleNewBlockActual` function, the settlement polling mechanism requests settlements with a `Limit` of `100` and sets `Reverse` to `true`, which instructs the Cosmos query to return the newest 100 settlements first.

Because the Cosmos SDK stores all settlements without marking processed ones as succeeded, the node repeats the same batch of recent entries on every iteration rather than advancing to older ones. Older settlements will therefore remain unhandled indefinitely, blocking the settlement pipeline.

Recommendation

We recommend fetching settlements in chronological order so that the oldest unprocessed items are handled first and marking processed settlements as completed to prevent reprocessing.

Status: Acknowledged

The client notes that it will be addressed as part of broader rearchitecting efforts planned for the system.

5. Refund logic allows already fulfilled requests to be reimbursed, allowing double spends and prematurely refunded deposits, resulting in locked liquidity

Severity: Critical

In `xarchain:x/chainabstraction/keeper/msg_server_refund_req.go:14-66`, the `RefundReq` message handler returns the deposited assets once the request-for-funds (rff) has expired. The handler relies on the `rff.Fulfilled` flag in line `33` to reject refunds that have already been satisfied.

However, the flag is populated by the vote-extension pipeline, which updates the chain state only after it observes the fulfillment transaction on the external destination chain. When the fulfillment transaction is executed shortly before expiry, there is a critical time window in which the external chain has already paid out the request, but `rff.Fulfilled` has not been set to `true`. During that window, the requester can call `RefundReq` to reclaim the deposited funds, receiving both fulfillment on the external chain and a refund from the vault. This enables a double-spend of the bridge's liquidity, resulting in a direct loss of funds.

Similarly, when the RFF expired without a successful fulfillment, the message handler refunds deposits on the source chain that were observed as `RFFSourceState_DEPOSITED`. If all deposits have been executed on the source chains but validators have not yet reached consensus via vote extension across all of them, the handler refunds all confirmed deposits and leaves the unconfirmed deposit locked in the vault. Because `rff.Refunded` becomes `true`, the request cannot be retried, and the liquidity remains stranded.

The combined effect results in losses from double refunds and permanently locked liquidity whenever on-chain activity lags behind verification.

Recommendation

We recommend ensuring that refunds are processed only after the protocol has deterministically confirmed whether the RFF deposits and fulfillment succeeded or failed. Possible mitigations include tracking request-for-funds in transit and requiring additional confirmation before issuing refunds, or implementing a longer delay between fulfillment and expiry to allow state synchronization across chains.

Status: Partially Resolved

The client notes that the current 2-minute delay is a known configuration that will remain in place for the time being. It will be revisited as part of broader rearchitecting efforts planned for the system.

6. Unvalidated RFF publication enables P2P network spam and resource exhaustion

Severity: Major

In the `publishRFFToNewRequestsTopic` function in `vservice/rff.go:171-186`, RFF IDs are published to the P2P network without validation. The function accepts any RFF ID and publishes it to the GossipSub topic without verifying whether the RFF exists on-chain, is valid, or has already been fulfilled.

While solvers validate the RFF's existence after receiving the message, the issue occurs during propagation: each message is broadcast to all P2P peers, consuming network bandwidth, flooding message queues, and triggering processing on every solver node. Even with early validation, each solver must perform Cosmos queries and spawn goroutines to verify the RFF, consuming computational and network resources. An attacker can spam arbitrary RFF IDs to exhaust P2P network resources, overwhelm solver nodes, consume the CA service's gas tokens during RFF processing, and potentially cause denial-of-service for legitimate RFF processing.

Recommendation

We recommend validating RFF IDs before publishing by checking on-chain existence, fulfillment status, and expiry. Additionally, implement rate limiting, authentication, and duplicate detection to prevent abuse.

Status: Acknowledged

The client acknowledges this finding with the note that VSCs will handle rate-limiting at the HTTP layer via a reverse proxy. They further clarify that since the P2P mesh is inherently open to the public and allows anyone to publish messages to the topic, rate-limiting the `/api/v1/publish-rff` endpoint alone would not provide meaningful protection against abuse.

7. Unbounded pending settlements are processed during the `PreBlocker`, which can cause consensus delays

Severity: Major

In `xarchain:x/chainabstraction/abci/proposal.go:168-230`, the `PreBlocker` function calls `keeper.GetAllPendingSettlement` every `SettlementInterval` and aggregates every pending settlement entry into a settlement struct before re-enqueuing them through `keeper.AppendSettlement`.

However, the pending settlements are unbounded. If there are many pending settlements, a proposer must process them, which can take considerable time and potentially exceed block timeouts, delaying block production.

Recommendation

We recommend enforcing a hard upper bound or a batching mechanism on pending settlements per interval (e.g., by limiting queue length, rate-limiting settlement creation, or processing settlements across multiple blocks) so that PrepareProposal cannot be forced to iterate over an unbounded set.

Status: Acknowledged

The client acknowledges this finding with the note that the settlement aggregation process is purely CPU, RAM, and local I/O bound. They contend that reaching a volume of pending settlements large enough to materially slow down block production on a block-to-block basis is practically infeasible under realistic operating conditions.

8. External RPC dependencies in consensus logic allow validators to be stalled

Severity: Major

In `xarchain:x/chainabstraction/keeper/msg_server_request_for_funds.go:188`, the `CreateRequestForFunds` message handler invokes `VerifyRFFSignature`, which is implemented in `x/chainabstraction/evmapi/evmapi.go:73-88`. That routine interacts with an external RPC that communicates with another blockchain (e.g., Ethereum, FUEL). Any slowdown, outage, or inconsistent response from that RPC endpoint prevents the transaction from completing and risks diverging results across validators.

The same dependency appears in `x/chainabstraction/keeper/msg_server_double_check_tx.go:38-54`, where `MsgDoubleCheckTx` passes vote-extension packets into `VerifyVEPacket`, implemented in `xarchain:x/chainabstraction/keeper/ve_verifier.go:55-70`. Inside `VerifyVEPacket`, both `HasWitnessedRequest` and `VerifyFillerAddress` functions perform further live external contract calls.

Finally, `VerifyVoteExtensionHandler` in `xarchain:x/chainabstraction/abci/vote_extension.go:187-229` reuses `VerifyVEPacket` when checking vote extensions during consensus. These executions are consensus-critical, they run on every validator, must be deterministic, and block progress while waiting for the external RPC. An attacker who rate-limits or corrupts responses from any of the referenced RPC providers, or an accidental provider outage, can therefore halt the inclusion of the affected messages, cause vote extensions to be rejected, and ultimately stall block production.

Recommendation

We recommend removing external RPC dependencies from consensus-critical paths. Signature and state checks should either rely solely on locally available data, be reproduced within the chain via pre-verified proofs, or be performed off-chain before submission, so validators no longer depend on live RPC calls during message handling or vote extension verification.

Status: Acknowledged

The client acknowledges this finding with the note that the decision not to perform local signature verification is intentional and driven by various design considerations. They further state that this concern is partially mitigated through system design and other layers within the architecture.

9. Missing minimum block confirmation margin allows external reorgs to invalidate verified requests

Severity: Major

In `xarchain:x/chainabstraction/keeper/ve_verifier.go:52-70`, the `VerifyVEPacket` function calls `HasWitnessedRequest` for deposit packets and `VerifyFillerAddress` for fulfilment packets to verify whether the request has been executed on the external destination chain.

However, the implementation does not require a minimum block confirmation depth or a finalized milestone on the external chain before marking a request as valid. If a short-lived reorganization occurs on the external chain (for example, on Polygon), the previously observed event may disappear while the `chainabstraction` module continues treating the request as settled, enabling incorrect fulfillments or releasing funds without a corresponding on-chain event. This exposure allows an adversary to exploit transient forks to satisfy off-chain settlement requirements that the canonical chain never confirms, resulting in a loss of funds for the protocol.

Recommendation

We recommend requiring a minimum number of block confirmations or a finalized milestone on the external chain.

Status: Acknowledged

The client acknowledges this finding, noting that 1-block confirmation was an intentional design choice and that the associated risks were intended to be communicated in the overall system design documentation.

However, they concur that while 1-block confirmations may be acceptable on chains like Ethereum Mainnet, chains such as Polygon experience more frequent reorgs, making this assumption unsafe across all supported networks. They further state that this will be addressed in the rearchitected system.

10. Fetching vote extension events without timeouts can stall consensus

Severity: Major

In `xarchain:x/chainabstraction/abci/vote_extension.go:136`, the `ExtendVoteHandler` function launches a goroutine per external chain that invokes `FetchEvents` without ever applying a timeout. Inside `FetchEvents`, calls such as `chain.Interface.GetRelevantLogsInBlockRange` executes against external RPC providers without a deadline, so a slow or unresponsive endpoint can block the goroutine. Because the handler waits on an `errgroup` for every goroutine to finish, a single blocked

request can prevent the ABCI method from returning its vote extension in time, preventing cross-chain bridge intents from being included in blocks.

Recommendation

We recommend enforcing a timeout for fetching events from external chains.

Status: Resolved

11. Unsupported settlement chain stops CA processing for all settlements

Severity: Major

In the `handleNewBlockActual` function in `ca-node:vservice/settlement_ev.go:60`, the code returns an error when a settlement references a chain identifier that is not present in the node's local registry. Because the Cosmos-side accepts arbitrary chain identifiers, and other solvers might support a larger variety of chains, it is possible to have settlements for chains that a given solver does not support.

Once such a settlement is encountered, the function exits early, and the `handleNewBlockActual` function aborts the entire batch, which leaves every other settlement in the same response unprocessed. The offending settlement remains in the on-chain list, so subsequent blocks will continue to trigger the same failure, effectively halting settlement processing for that solver.

Recommendation

We recommend replacing the early return with logic that skips unsupported settlements, for example, by continuing the loop after logging the condition so the solver can keep processing the remaining valid settlements.

Status: Resolved

12. Disabling libp2p limits enables remote DoS

Severity: Major

In `ca-node:internal/p2p/p2p.go:52`, the `CreateNewP2PSetup` function instantiates the libp2p resource manager with `rcmgr.NewFixedLimiter(rcmgr.InfiniteLimits)`, which disables all accounting of streams, memory, and file descriptors. When the limits are infinite, the resource manager cannot enforce per-peer or system-wide caps, so any connected counterparty can spawn unbounded streams or request excessive buffers.

Although the connection manager still bounds the number of peers, a single malicious peer can monopolize memory or descriptors until the process exhausts host resources. This enables a remote denial-of-service condition against every node that adopts this configuration.

Recommendation

We recommend configuring the libp2p resource manager with finite, environment-appropriate limits (for example, by using `rcmgr.InfiniteLimits.ToPartialLimitConfig().Build(...)` and overriding critical ceilings) so that abusive peers are throttled before they can exhaust local resources.

Status: Resolved

13. Transient DSG send failures slip through retries and stall key operations

Severity: Major

In `ca-node:vservice/dsg.go:249-256`, the closure passed to the retry helper returns the outer `err` variable when `a.p2pSend` fails, even though that variable remains `nil`. Because the retry loop interprets a `nil` return value as success, libp2p transmission failures do not trigger another attempt, and the error never propagates to callers.

As a result, in addition to logging the incorrect `err` variable, any temporary network disruption causes the sending node to drop critical DSG messages, halting distributed key generation and signing across the cluster.

Recommendation

We recommend returning the actual `err2` value from the closure, logging it for visibility, and allowing the retry backoff to execute.

Status: Resolved

14. Replayed DSG messages crash CA nodes through closed channel writes

Severity: Major

In `ca-nodevservice/dsg.go:119-163`, the `setupDSGStreamHandler` function accepts any correctly signed DSG payload and forwards it to `MessageKeeper` without freshness or nonce checks. Because the handler does not enforce replay protection, an allowlisted malicious participant can resend any previously valid DSG message at will. In `vservice/msgkeeper.go:85-96`, the `ProcessMsg` function closes the round's channel

once the expected message count is reached but retains the round entry in `MessageKeeper.rounds`.

Therefore, a replayed payload still passes the check in the `HasRound` function, and `ProcessMsg` attempts to enqueue it on the closed channel, which triggers a `panic: send on closed channel`, causing the service to crash.

Recommendation

We recommend removing the round from `MessageKeeper.rounds` immediately after its channel is closed.

Status: Resolved

15. Incomplete fee grant implementation

Severity: Major

In `ca-node:vservice/feegrant.go:30-36`, the `NewMsgGrantAllowance` function creates a fee grant with hardcoded placeholder values that are not production-ready. Specifically, the allowance uses a test denom "`test`" with an amount of `1`, and sets `Expiration` to nil, meaning the grant never expires.

Additionally, the `createFeeGrant` entry point is publicly accessible with no access controls or rate limiting. This allows any user, including external users who do not use the protocol, to repeatedly call this endpoint to receive unlimited fee grants and avoid paying transaction fees.

Recommendation

We recommend implementing the following changes before production deployment:

- Replace the hardcoded "`test`" denom with the actual denom to be granted for allowance
- Implement a proper expiration timestamp instead of setting `Expiration` to `nil`
- Add rate limiting to the `createFeeGrant` endpoint to prevent spam, for example, consider implementing per-user limits or cooldown periods for fee grant requests

Status: Acknowledged

The client acknowledges this finding with the note that the Cosmos chain operates as a feeless network, and the fee-grant mechanism exists solely to initialize the account sequence. They recognize the potential denial-of-service vector and intend to address it through alternative mitigations outside the scope of this particular logic.

16. Switching the connected wallet continues to use the previous EVM address

Severity: Major

In `nexus-sdk:packages/core/sdk/ca-base/ca.ts:416`, the `createCosmosWallet` function retrieves and persists the cached SIWE signature under `this._evm.address`. That field is only populated by the `_setEVMProvider` function in lines `293-307`, which is only called once during `initialize`, while the `accountsChanged` hook in lines `286-290` merely re-initializes the stack without refreshing `this._evm.address`.

After the provider reports an account change, the SDK continues to index SIWE signatures, Cosmos wallet derivation, and the bridge options with the previous account. As a result, downstream flows such as balance queries, allowance checks, refund scans, and bridge intent execution run against the wrong address, even though the wallet extension now exposes a different account. This creates the risk that value-moving operations or approvals are applied to the wrong account, preventing the newly selected account from interacting reliably.

Recommendation

We recommend updating `this._evm.address` whenever the provider's `accountsChanged` event fires (for example, by reusing the `_setEVMProvider` function with the current account) and ensuring SIWE signatures, Cosmos wallet derivation, and bridge handlers always reference the freshly selected address.

Status: Resolved

17. Compromised SIWE signature enables permanent theft of derived Cosmos and ephemeral wallets

Severity: Major

In `nexus-sdk:packages/core/sdk/ca-base/ca.ts:415-427`, the `_createCosmosWallet` function derives both the Cosmos signer and the Calibur "ephemeral" EVM account from a SIWE signature obtained once per EVM address. That signature is stored in browser storage, and the SIWE message hardcodes `nonce` and `issuedAt`.

As a consequence, any script that can read from local storage, including injected XSS, malicious dependencies, or another extension on the same origin, can exfiltrate the cached signature and deterministically reconstruct the same private key pair indefinitely. This yields full control over the delegated EVM wallet, the Cosmos account, existing intents, and any future Calibur batched calls without further user interaction.

Recommendation

We recommend eliminating the static SIWE seed and avoiding plaintext storage of reusable signing material. A safer pattern is to remove the hardcoded nonce and timestamp, derive the key once, and persist the resulting secret only in encrypted form (e.g., using WebCrypto with a user-provided passphrase or a hardware-backed WebAuthn credential).

Status: Acknowledged

The client acknowledges this finding with the note that any party with access to the EIP-1193 provider can prompt the user to re-sign the SIWE message, a routine user interaction, and thereby re-derive the same private key.

They further state that guarding against adversaries capable of compromising running code within the JavaScript context, including access to storage, RAM, or the provider itself, is not feasible. Since an app developer could already maliciously abuse users regardless of this design choice, the finding does not introduce additional trust assumptions. Additionally, because SIWE signatures are domain-bound, a compromised derived key does not affect other applications.

18. `CreateProtocolFees` lacks an authorization check, allowing anyone to set themselves as admin

Severity: Major

In the `CreateProtocolFees` function in `xarchain:x/chainabstraction/keeper/msg_server_protocol_fees.go:13-35`, there is no authorization check to verify that the caller is authorized to create protocol fees. The function only checks whether protocol fees already exist in `lines 17-20`. If they don't, any user can call this function to set themselves as the `protocolFees` admin, set arbitrary fee percentages, and configure fee recipients. This allows any attacker to be the first to call `CreateProtocolFees` after the chain launch, thereby gaining complete control over protocol fees.

Recommendation

We recommend adding an authorization check similar to `UpdateProtocolFees` or `MsgUpdateParams`, such as checking that `msg.Creator` matches the module authority `k.GetAuthority`, requiring this to be a governance proposal, or checking against a predefined admin address set during genesis initialization.

Status: Resolved

19. Signature verification failure enables zero solver addresses and burns settlements

Severity: Minor

In `xarchain:x/chainabstraction/evmapi/evmapi.go:90-107`, the `VerifyGenericMessageSignature` function returns `universeabstraction.ZeroCommonAddress` together with a `nil` error whenever `crypto.SigToPub` fails to recover a public key. The caller function `validateSolverDataAddresses` in `xarchain:x/chainabstraction/keeper/msg_server_solver_data.go:22-61` only checks the error value before comparing the supplied address with the returned signer. Because both values are zero, a solver that accidentally submits the zero address together with an invalid signature is accepted.

Downstream, the settlement flow in `ExecuteVE` in `xarchain:x/chainabstraction/keeper/ve_exec.go:56-187` records this zero address as the payout recipient, so the protocol schedules settlements to `0x000…0`. Those transfers are unrecoverable, resulting in a direct loss of funds for the solver and breaking settlements for that request.

Recommendation

We recommend propagating the signature recovery error (or returning a dedicated failure) from `VerifyGenericMessageSignature` and rejecting zero addresses explicitly in the solver-data validation so that settlement recipients cannot resolve to the zero address.

Status: Resolved

20. Using `log.Fatal` can halt the validator

Severity: Minor

In `xarchain:x/chainabstraction/evmapi/evm_logs.go:37`, the `GetRelevantLogsInBlockRange` function calls `log.Fatal` when the vault ABI fails to parse, and in lines `68` and `78`, the same function invokes `log.Fatal` when unpacking the `Deposit` and `Fulfilment` logs. Because `log.Fatal` terminates the process through `os.Exit(1)`, it forces the Cosmos SDK node to stop rather than returning an error through the module's consensus-safe path. This turns external log data into a validator-wide denial-of-service, requiring operator intervention to restart and leaving the validator unable to participate in consensus while the fault persists.

Recommendation

We recommend replacing the `log.Fatal` calls with error returns (or module-level logging) so that parsing failures propagate through the Cosmos SDK error handling path, allowing the validator to report the failure without terminating.

Status: Acknowledged

The client acknowledges this finding with the note that the ABI data is hardcoded and cannot return an error in practice under normal operation. Consequently, the unhandled error path is considered harmless.

21. Known vulnerable Go dependencies

Severity: Minor

In `xarchain:go.mod` and `ca-node:go.mod`, a large set of direct dependencies is pinned, and running `govulncheck ./...` reports several outstanding advisories for the current dependency graph. Each flagged advisory indicates that the project links against code containing publicly disclosed vulnerabilities, so deploying the current build leaves the binary exposed to those attack surfaces.

Recommendation

We recommend upgrading any dependencies reported as vulnerable.

Status: Resolved

22. Gas refund flag in deposit packet is ignored by the Cosmos chain

Severity: Minor

In `ca-node:vservice/rff.go:129-133`, the service constructs `MsgDoubleCheckTx` packets with a `DepositVEPacket` payload that sets `GasRefunded` to `true`. However, the Cosmos application processes the same packet type using a version that only exposes the `Id` field, so the gas refund flag is silently discarded when the message is decoded, leading to potential mismatches in expected behavior between the CA service and the chain.

Recommendation

We recommend updating both the service and the Cosmos module to share an identical protobuf definition for `DepositVEPacket`.

Status: Resolved

23. Default price of 1.0 for unknown tokens

Severity: Minor

In `ca-node:vservice/unified_bal.go:45-47`, when `GetPriceForCurrencyFromPOD` returns `ErrInvalidCurrency` because a token's price is not found in the price oracle data, the code defaults the price to 1.0 by setting `price = apd.New(1, 0)`. This means any token without a registered price is treated as having a real-world value of 1.0 USD.

This causes the unified balance report to display inflated and inaccurate values. Users holding large quantities of worthless tokens would see their total balance incorrectly reported as significant USD values, misrepresenting their actual holdings.

We classify this issue as minor, as it affects data accuracy and user experience but does not directly compromise protocol security or result in a loss of funds.

Recommendation

We recommend changing the default behavior when a token price is not found. Instead of defaulting to `1.0`, the code should either return an error to indicate that the balance cannot be accurately calculated or default to 0 to avoid inflating balance reports with worthless tokens.

Status: Resolved

24. Signed batched call signatures never expire, enabling replay when leaked

Severity: Minor

In `nexus-sdk:packages/core/sdk/ca-base/swap/sbc.ts:24-51`, the `createBatchedCallSignature` function encodes the `deadline` field of the `SignedBatchedCall` payload as `maxUint256`. The same hardcoded value is propagated when the serialized request is assembled in lines `73-146` via the `createSBCTxFromCalls` function.

Because these signatures never expire, any signed payload that leaks prior to on-chain submission remains executable indefinitely until its nonce is consumed. An adversary who gains access to such a signature can execute the batched call at any time, even after the user's intent has changed, enabling the unauthorized execution of stale operations and potentially resulting in asset transfers or other unwanted state changes.

Recommendation

We recommend exposing a short, caller-configurable deadline for batched call signatures.

Status: Resolved

25. Params validate performs no validation, allowing invalid parameters to be stored

Severity: Minor

In the `Validate` function in `xarchain:x/chainabstraction/types/params.go:30-32`, the function always returns nil without validating any fields. The Params struct contains sensitive configuration, including `Universe` enum values, `ChainID` byte slices, `DisabledTokens` byte slices, and other nested configs. This allows governance proposals or genesis initialization to set invalid or malformed parameters.

Recommendation

We recommend implementing comprehensive validation in the `Validate` function to check that `UniverseParams` are valid.

Status: Acknowledged

The client acknowledges this finding with the note that this parameter is only configurable via a governance proposal, making additional validation generally unnecessary. They may consider adding explicit validation in a future update as a defensive measure.

26. Protocol fees messages lack validation for fee configuration fields

Severity: Minor

In the `ValidateBasic` functions in `xarchain:x/chainabstraction/types/messages_protocol_fees.go:11-17` and `21-26`, only the `Creator` address is validated, while all other fields are accepted without validation. This is problematic because these functions may introduce critical misconfiguration issues if invalid parameters are passed.

Recommendation

We recommend implementing comprehensive validation including checking that `FeeBP` does not exceed 10000 (100%), validating that `Universe` enum values are valid, validating `ChainID` and `TokenAddress` byte slices have correct lengths for their respective chains, validating `Fee` byte slices are properly formatted and represent valid amounts, validating `Address` byte slices in `AdminFeeRecipient` have correct lengths, checking for duplicate entries in arrays, and ensuring all required fields are present and non-empty where appropriate.

Status: Resolved

27. Service startup might hang due to an infinite peer connection retry loop

Severity: Minor

In the `main` function in `ca-node:vservice/main.go:130-141`, a peer ping loop retries indefinitely with no timeout or cancellation. The code waits for a response from each peer in `a.dsgNodeDetails.nodes` and retries every 10 seconds if a response is not received. If a peer never responds, the startup blocks indefinitely, resulting in a denial-of-service and preventing the application from starting.

Recommendation

We recommend adding a timeout mechanism, such as a context with a deadline, a maximum retry count, or a timeout for the entire peer connection phase, to ensure startup completes even if some peers are unavailable.

Status: Resolved

28. DSG threshold set to 100%

Severity: Minor

In the `main` function in `ca-node:vservice/main.go:107`, the DSG threshold is hardcoded to equal the total number of participants, requiring all nodes to participate in distributed operations. This means any single node failure or unavailability creates a single point of failure. The system cannot tolerate even a single offline or malicious node, making it vulnerable to denial-of-service attacks and operational failures.

Recommendation

We recommend making the threshold configurable and setting it to a value that provides Byzantine fault tolerance, such as (n/2 + 1) for n nodes, allowing the system to function with up to (n-1)/2 nodes being unavailable or malicious.

Status: Resolved

29. Potentially non-deterministic nonce derivation

Severity: Minor

In the `createSBCTxFromCalls` function in `nexus-sdk:packages/core/sdk/ca-base/swap/sbc.ts:100-110`, the authorization nonce is fetched using `getTransactionCount` with `blockTag: 'pending'` without synchronization. If multiple SBC transactions are created concurrently, or within a larger window before the next transaction is confirmed, multiple SBC transactions may attempt to use the same nonce, and the latter will fail.

Recommendation

We recommend implementing deterministic nonce derivation in the `createSBCTxFromCalls` function.

Status: Resolved

30. Missing disabled vote extension check

Severity: Informational

In `xarchain:x/chainabstraction/abci/proposal.go:111`, the `PreBlocker` function checks whether to process vote extensions from the condition `req.Height > ctx.ConsensusParams().Abci.VoteExtensionsEnableHeight`. The issue is that it does not also verify that vote extensions are enabled (i.e., by checking that `VoteExtensionsEnableHeight` is non-zero).

According to CometBFT documentation, once vote extensions are enabled, they cannot be disabled. Therefore, in the intended operation of the protocol, where vote extensions are enabled at the beginning, this inconsistency does not pose a material risk.

The issue also affects the `ProcessProposal` function in `x/chainabstraction/abci/proposal.go:78`.

Recommendation

We recommend adding validation to ensure that `VoteExtensionsEnableHeight` is non-zero before processing vote extensions in the `PreBlocker` and `ProcessProposal` functions.

Status: Resolved

31. Multiple messages return empty responses

Severity: Informational

In the UpdateParams function in `xarchain:x/chainabstraction/keeper/msg_update_params.go:22`, the function returns an empty `MsgUpdateParamsResponse` struct with no fields populated. It is best practice to return response attributes that reflect the actions and state changes performed.

The following chain abstraction message handlers also return empty responses:

- `DoubleCheckTx` in `xarchain:x/chainabstraction/keeper/msg_server_double_check_tx.go:57`
- `CreateSolverData` in `xarchain:x/chainabstraction/keeper/msg_server_solver_data.go:91`
- `UpdateSolverData` in `xarchain:x/chainabstraction/keeper/msg_server_solver_data.go:122`
- `CreateProtocolFees` in `xarchain:x/chainabstraction/keeper/msg_server_protocol_fees.go:34`
- `UpdateProtocolFees` in`xarchain:x/chainabstraction/keeper/msg_server_protocol_fees.go:61`
- `UpdateParams` in`xarchain:x/chainabstraction/keeper/msg_update_params.go:22`
- `RefundReq` in `xarchain:x/chainabstraction/keeper/msg_server_refund_req.go:66`

Recommendation

We recommend adding meaningful response fields to the responses mentioned above that accurately reflect the state changes in their respective messages.

Status: Acknowledged

32. Missing validation for RPC URLs and vault contract addresses during initialization

Severity: Informational

In the chain initialization loop in `xarchain:x/chainabstraction/keeper/keeper.go:452-465`, the code validates that RPC URLs exist and are non-empty in lines `452-460` and that vault contract addresses exist in lines `461-465`, but it does not validate URL format, connectivity, or contract address format/validity.

While malformed RPC URLs will eventually cause panics when `ethclient.Dial` is called in `evmapi.NewEVMAPI` in `line 41`, and invalid contract addresses may cause failures during contract interactions. These errors occur after the initial validation checks pass. Since these values are set during chain initialization and are unlikely to be incorrect, the risk is low. However, configuration errors may not be detected until runtime operations fail.

Recommendation

We recommend adding validation during initialization: for RPC URLs, perform a simple connectivity check such as calling `eth_chainId` for EVM chains; for vault contract addresses, perform a simple contract query to verify the address is valid, and the contract exists. This would provide early detection of configuration issues, though given these are init-time configurations, the benefit may be limited.

Status: Acknowledged

33. Hardcoded gas limits may cause inefficiency or failures

Severity: Informational

In the `createFeeGrant` function in `ca-node:vservice/feegrant.go:40`, the gas limit is hardcoded to `3_00_000` when submitting the fee grant transaction. This fixed value may not match the actual gas required for the transaction.

In addition, this issue is also present in `ca-node:vservice/rff.go:142`.

Recommendation

We recommend making the gas limits configurable and then having a fallback value.

Status: Acknowledged